

Implementasi Metode *YOLOv8n* dan *Canny Edge Detection* Untuk Sistem Deteksi Objek *Intrusion* dan *Trespasser* Pada Jalur Kereta Api

Moh. Farhan Baihaqi 224308037 – TKA 6B
Politeknik Negeri Madiun
Madiun, Indonesia
Farhanbaihaqi2003@gmail.com

Farhat Fadel Yusmanan 224308033 – TKA 6B
Politeknik Negeri Madiun
Madiun, Indonesia
Farhatfadel17@gmail.com

Axl Aditya Persada 224308029 – TKA 6B
Politeknik Negeri Madiun
Madiun, Indonesia
axlmadiun@gmail.com

M Daffa Adyatma P. 2243080238 – TKA 6B
Politeknik Negeri Madiun
Madiun, Indonesia
moch.daffajr@gmail.com

Abstract— *Intrusion* and *Trespasser* detection pada jalur kereta api menggunakan metode *Canny edge detection* berbasis *YOLOv8-n*. Pengujian ini dilakukan karena keberadaan *intrusion* (benda mati seperti batu, pohon tumbang, atau material konstruksi) dan *trespasser* (manusia atau makhluk hidup lain yang melakukan aktivitas) di jalur kereta api dapat menimbulkan risiko besar terhadap keselamatan perjalanan kereta api. Dengan melakukan pengujian menggunakan metode *YOLOv8n*, yang mampu mendeteksi dan mengklasifikasikan objek *intrusion* dan *trespasser* secara langsung dari citra dalam satu tahap proses dan metode *canny edge detection* digunakan untuk memperjelas kontur atau batas objek, sehingga membantu dalam membedakan bentuk *intrusion* dan *trespasser*, kemudian dalam pengujian ini mendapatkan nilai rata-rata akurasi sebesar 68,56%. Kesimpulan dari pengujian ini adalah memberikan kinerja model dalam mendeteksi objek pada jalur kereta api menggunakan 2 metode sehingga membantu keselamatan dalam bidang transportasi kereta api.

Keywords—*Canny Edge Detection*, *YOLOv8*, *Intrusion*, *Trespasser*, *object detection*, *jalur kereta api*.

I. INTRODUCTION

Perkembangan Teknologi *artificial intelligence* menjadi hal yang penting di negara-negara maju dan berkembang pada saat ini. *Artificial Intelligence* merupakan bidang ilmu computer yang digunakan untuk pembuatan *software* dan *hardware* yang dapat bekerja seperti kecerdasan manusia. AI digunakan dalam proses otomatisasi seperti machine learning, natural language processing dan computer vision untuk pengolahan data, untuk mendeteksi object, pengolahan citra. Teknologi ini telah mengalami kemajuan pesat berkat perkembangan *deep learning* dan *Convolutional Neural Network* (CNN) [1].

Computer vision saat ini banyak digunakan di bidang transportasi yaitu kereta api, yang digunakan sebagai deteksi objek. Deteksi objek merupakan proses mengenali dan menentukan posisi objek yang terdapat pada sebuah gambar atau video. Algoritma yang diterapkan dalam proses deteksi objek menggunakan metode pembelajaran mesin mendalam untuk menghasilkan hasil yang optimal. Deteksi objek memungkinkan sistem untuk secara otomatis dan efisien

mengenali dan menandai objek dalam berbagai konteks visual [2].

Pada transportasi kereta api deteksi objek digunakan pada jalur kereta api. Di sepanjang jalur rel kereta api harus aman dari benda atau makhluk hidup yang tidak seharusnya berada di jalur kereta api, yang berpotensi menyebabkan kecelakaan kereta api. Benda atau makhluk hidup yang berada pada jalur rel memiliki dua klasifikasi yaitu *intrusion* dan *trespasser*.

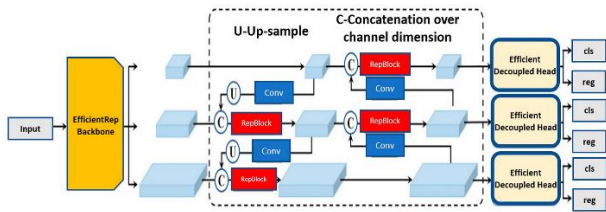
Intrusion merupakan masuknya benda yang masuk ke daerah aman di jalur kereta api, *intrusion* berupa benda mati seperti: batu, pohon tumbang, material konstruksi [3]. Sedangkan *trespasser* merupakan aktivitas yang dilakukan diatas atau disekitar rel tanpa izin, sehingga membahayakan keselamatan dan mengganggu operasional perjalanan kereta api [4]. Berdasarkan permasalahan hal tersebut dilakukan deteksi objek untuk mendeteksi benda (*intrusion*) atau makhluk hidup (*trespasser*) yang berada pada jalur kereta api menggunakan 2 metode yaitu *YOLOv8n* dan *canny edge detection* yang bisa digunakan untuk mengurangi kecelakaan kereta api dan meningkatkan keselamatan perjalanan kereta api [3]

Pengujian ini dilakukan menggunakan metode algoritma *YOLOv8n* dengan *transfer learning* dalam pelatihan untuk menghasilkan model [5]. Model *YOLOv8n* dapat mendeteksi *intrusion* dan *trespasser* pada jalur kereta api yang melibatkan langkah-langkah berikut: pertama, pengumpulan dataset citra yang kemudian dilakukan pelabelan atau anotasi pada dataset citra dengan kelas atau label. Kedua, pra-pemrosesan dan augmentasi citra dilakukan untuk mempersiapkan dataset agar sesuai dengan format yang diperlukan oleh algoritma YOLO dan dataset yang digunakan dibagi menjadi tiga bagian data yaitu data pelatihan (train), data validasi (valid) dan data pengujian (test). Lalu melakukan pengujian deteksi objek *intrusion* dan *trespasser* pada jalur kereta api yang telah dimanipulasi sehingga dapat mengukur performa model dalam mendeteksi objek pada citra [2].

Pengolahan citra merupakan proses yang melibatkan analisis dan pengolahan citra dengan mempertimbangkan persepsi visual [6]. Metode *canny edge detection* merupakan salah satu teknik dalam pengolahan citra yang memiliki pengaruh besar karena kemampuannya dalam mendeteksi tepi dengan efektif. Teknik ini memanfaatkan serangkaian algoritma yang terdiri dari beberapa langkah, termasuk penghalusan gambar untuk mengurangi noise, perhitungan gradien untuk menentukan kekuatan dan arah tepi, penekanan non-maksimum untuk mempertajam tepi, serta penghubungan tepi dengan histeresis untuk mengidentifikasi tepi yang kuat dan lemah [7]. Hasil dari pengujian ini adalah garis-garis tepi yang jelas yang selanjutnya dapat digunakan untuk mengenali karakteristik dari dua objek klasifikasi *intrusion* dan *trespasser*.

II. METHOD

Pengujian ini menggunakan algoritma YOLOv8n untuk mendeteksi objek *intrusion* dan *trespasser*. You Only Look Once (YOLO)v8n adalah salah satu algoritma object detection yang dapat mendeteksi objek pada gambar atau video secara real-time [8]. Secara garis besar YOLO membagi gambar input menjadi suatu grid dengan sel-sel tertentu. Setiap sel dalam grid bertanggung jawab untuk memprediksi beberapa bounding box beserta probabilitas kelas objek yang mungkin terdapat di dalamnya [2]. Pada algoritma YOLOv8n merupakan evolusi dari versi-versi YOLO sebelumnya yang berdasarkan pada CNN. YOLOv8n ini menghasilkan model jaringan saraf dengan kemampuan representasi fitur yang lebih unggul, sekaligus memperkuat aliran gradien selama proses pelatihan [5]. Metode ini memiliki tingkat struktur model yang cukup besar dan dalam membuat proses pelatihan dan konvergensi memerlukan waktu lebih lama, serta membutuhkan sumber daya komputasi yang lebih tinggi dibandingkan dengan algoritma deteksi objek lainnya.



Gambar 1. Architecture YOLOv8n

Metode YOLOv8n ini menggunakan *transfer learning* yang digunakan untuk mempelajari informasi sebelumnya (*pre-trained*) yang akan dipelajari [5]. Metode ini memungkinkan model untuk melakukan inisialisasi dengan parameter yang telah terlatih, sehingga dapat mempercepat proses konvergensi dan meningkatkan performa deteksi *intrusion* dan *trespasser* sesuai dengan dataset.

3.1 Pengumpulan Dataset

Dataset pada model *object detection* merupakan kumpulan citra untuk diproses pada tahap pelatihan model nantinya. Dataset citra yang digunakan harus sesuai dengan permasalahan yang dihadapi. Pengujian ini menggunakan sistem deteksi objek *intrusion* dan *trespasser* yang digunakan pada jalur kereta yang dapat membantu saat inspeksi. Pengujian ini memanfaatkan platform Roboflow yang dapat membantu mempermudah proses pengumpulan dan persiapan dataset [5].

3.2 Preprocessing dan Augmentation

Anotasi dan pelabelan merupakan bagian dari preprocessing pada dataset dengan tujuan memberikan tanda pada data untuk menentukan titik-titik koordinat bounding box [x] [y] yang juga memiliki informasi berupa kelas [class] yang diberi nama sesuai dengan nama objek. Pengujian ini memiliki 2 jumlah kelas yaitu kelas *intrusion* dan kelas *trespasser*.

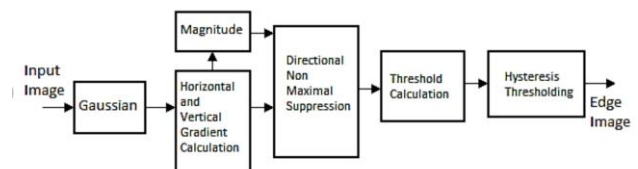
Augmentasi data adalah proses persiapan data yang memiliki tujuan untuk memperbanyak jumlah dataset sehingga meningkatkan jumlah variasi data. Augmentasi yang digunakan pada pengujian ini yaitu *Saturation (Between -25% and +25%)*.

3.3 Pelatihan Model YOLOv8n

Pada tahap ini, dilakukan pelatihan model menggunakan pada Google Colab meliputi pengecekan Google Colab, instalasi library yang diperlukan, impor dataset, konfigurasi hyperparameter YOLOv8n dan pelatihan. Pengecekan Google Colab dilakukan untuk melihat ketersediaan resources perangkat keras akselerator. Lalu instalasi library Roboflow untuk impor dataset dari platform Roboflow dan instalasi library Ultralytics untuk mengakses model YOLOv8 di dalamnya. Konfigurasi hyperparameter dilakukan dengan jumlah epochs = 100 dan imagesize = 640. Kemudian melakukan pelatihan model berdasar pada konfigurasi hyperparameter sebelumnya. Terakhir, penerapan model dijalankan pada platform Roboflow sehingga dapat dilakukan pengujian deteksi [5].

Kemudian, pengujian ini dilakukan menggunakan metode *Canny edge detection* untuk pengolahan citra (*image processing*) yang digunakan sebagai pendeteksi tepi objek dalam sebuah gambar. Gambar input dimasukkan ke dalam sistem, baik berupa gambar berwarna (RGB) maupun grayscale. Kemudian, gambar tersebut mengalami proses reduksi noise dengan menggunakan filter Gaussian Blur. Langkah ini untuk menghilangkan gangguan kecil yang dapat mengganggu deteksi tepi. Setelah itu, perhitungan gradien dilakukan dengan menggunakan operator Sobel, yang mendeteksi perubahan intensitas piksel di arah horizontal dan vertical [9]. Hasil dari perhitungan ini adalah nilai magnitude dan arah gradien, yang menunjukkan kekuatan dan arah tepi pada gambar. Selanjutnya, untuk menghasilkan garis tepi yang lebih tipis dan akurat, dilakukan proses Non-Maximum Suppression, yang mempertahankan hanya piksel-piksel yang merupakan nilai maksimum lokal di sepanjang arah gradien.

Setelah itu, dilakukan teknik *Thresholding Calculating*, yang mengklasifikasikan piksel menjadi tiga kategori berdasarkan intensitas gradiennya, yaitu tepi kuat (strong edge), tepi lemah (weak edge), dan bukan tepi (non-edge). Proses berikutnya adalah *Hysteresis Thresholding*, di mana piksel tepi lemah yang terhubung dengan tepi kuat akan dipertahankan, sementara yang tidak memiliki koneksi akan dibuang. Pada tahap akhir, sistem menghasilkan output berupa gambar biner yang hanya menampilkan garis-garis tepi dari objek yang ada di dalam gambar.

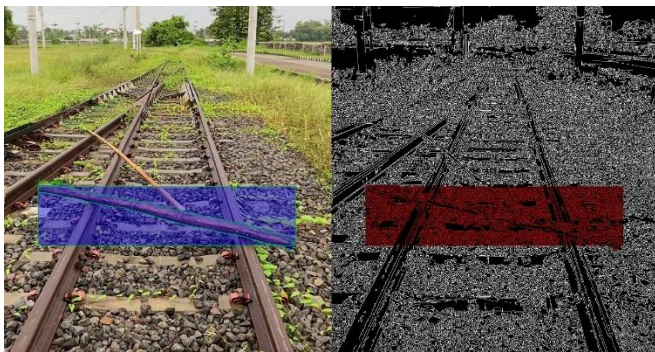


Gambar 2. Block Diagram Canny Edge Detection

Pada pengujian ini, setelah gambar diproses menggunakan metode *canny edge detection*, YOLOv8n digunakan untuk mendeteksi gambar berdasarkan kelasnya yaitu kelas *intrusion* dan kelas *trespasser* sehingga hasilnya berupa garis tepi sesuai dengan dataset yang sudah dilatih.

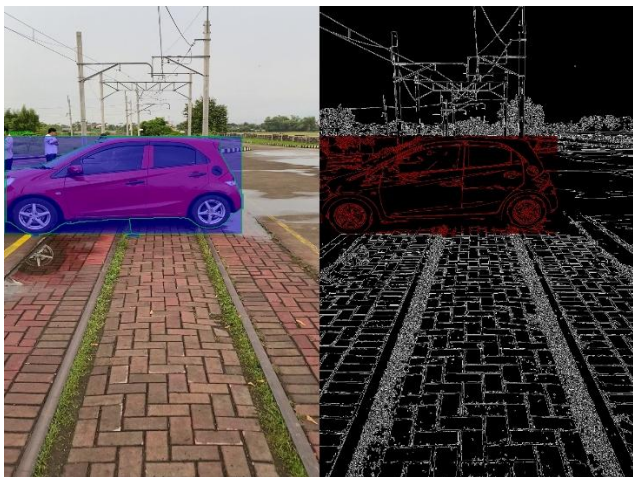
III. HASIL

Hasil pengujian ini menggunakan dataset yang sudah melewati proses pelatihan sebelumnya, sehingga dataset yang telah dilatih ini memungkinkan model untuk mengenali karakteristik objek *intrusion* dan *trespasser* secara lebih akurat, sehingga meningkatkan kinerja sistem dalam mendeteksi dan mengklasifikasikan objek pada citra uji. Hasil pengujian ini kemudian disajikan dalam bentuk dokumentasi dan tabel. Pada pengujian ini yang dapat dikatakan sebagai objek *intrusion* yaitu benda yang seharusnya tidak ada pada rel kereta api contoh (cone, kayu, helm, dan sebagainya), sedangkan objek yang bisa dikatakan sebagai objek *trespasser* yaitu objek yang boleh melintasi rel kereta api atau objek yang masih boleh berada di dekat rel kereta api, contoh (manusia, kendaraan bermotor, mobil).



Gambar 3. *Intrusion* = 1

Hasil gambar diatas menunjukkan kayu sebagai *intrusion*. Nilai hasil yang didapatkan speed: 8.0ms preprocess, 804.4ms inference, 8.1ms postprocess per image at shape (1, 3, 640, 384)



Gambar 4. *Trespasser* = 1

Hasil gambar diatas menunjukkan kendaraan roda empat sebagai *trespasser*. Nilai hasil yang didapatkan speed: 6.3ms preprocess, 800.1ms inference, 6.4ms postprocess per image at shape (1, 3, 640, 384)).



Gambar 5. *Trespasser* = 1

Hasil gambar diatas menunjukkan manusia sebagai *trespasser*. Nilai hasil speed: 11.7ms preprocess, 823.7ms inference, 7.9ms postprocess per image at shape (1, 3, 640, 384)



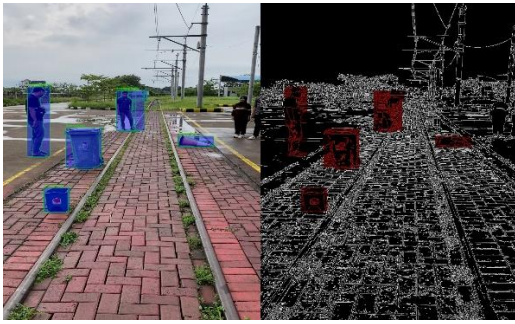
Gambar 6. *Intrusion* = 1 dan *Trespasser* = 3

Hasil gambar diatas menunjukkan manusia sebagai *trespasser* dan kayu sebagai *intrusion*. Nilai hasil yang didapatkan speed: 7.6ms, preprocess, 844.4ms inference, 10.8 ms postprocess per image at shape (1, 3, 640, 384).



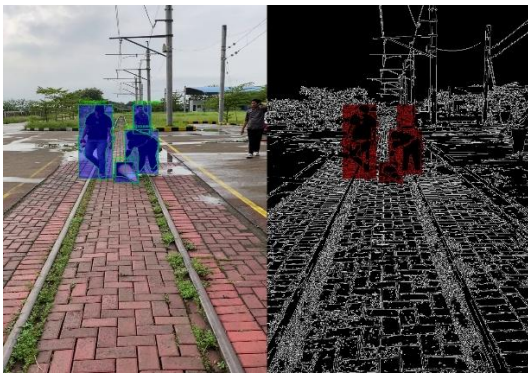
Gambar 7. *Intrusion* = 2 dan *Trespasser* = 1

Hasil gambar diatas menunjukkan manusia sebagai *trespasser*, cone dan tong sampah sebagai *intrusion*. Nilai hasil yang didapatkan speed: 9.7ms preprocess, 789.6ms inference, 11.8ms postprocess per image at shape (1, 3, 640, 384)



Gambar 8. *Intrusion* = 3 dan *Trespasser* = 2

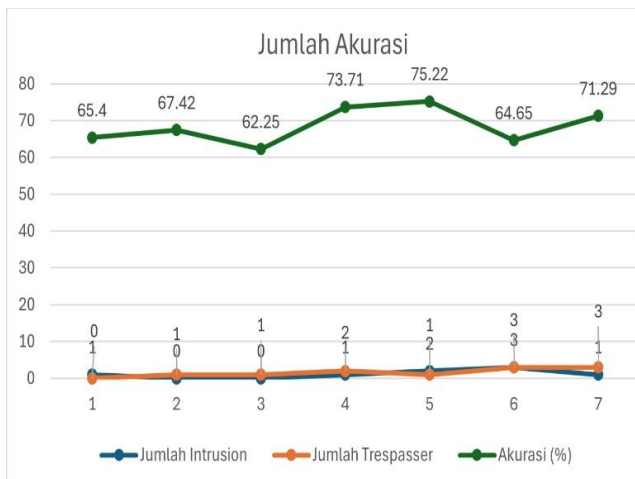
Hasil gambar diatas menunjukkan manusia sebagai *trespasser*, cone dan tong sampah sebagai *intrusion*. Nilai hasil yang didapatkan speed: 9.7ms preprocess, 1037.8ms inference, 19.6ms postprocess per image at shape (1, 3, 640, 384)



Gambar 9. *Intrusion* = 1 dan *Trespasser* = 3

Hasil gambar diatas menunjukkan manusia sebagai *trespasser* dan cone sebagai *intrusion*. Nilai hasil yang didapatkan speed: 5.7ms preprocess, 788.0ms inference, 14.4ms postprocess per image at shape (1, 3, 640, 384)

Selanjutnya, pengujian ini mendapatkan nilai akurasi. Nilai akurasi yang didapatkan dari menghitung rata-rata *confidence* dari seluruh prediksi yang dilakukan oleh model. *Confidence* menunjukkan tingkat keyakinan model terhadap hasil deteksi objek yang ditemukan. Semakin tinggi nilai *confidence*, maka semakin besar keyakinan model bahwa deteksi tersebut benar.



Gambar 9. Grafik *confidence score*

Nilai rata-rata *confidence* dihitung dengan membagi total *confidence* dari seluruh prediksi dengan jumlah total prediksi yang berhasil dilakukan. Hasil rata-rata ini kemudian dikalikan 100 untuk dinyatakan dalam bentuk persentase dan ditampilkan pada tampilan keluaran sebagai indikator performa deteksi objek.

NO.	Hasil Pengujian	Akurasi
1.	<i>Intrusion</i> = 1 dan <i>Trespasser</i> = 0	65,4%
2.	<i>Intrusion</i> = 0 dan <i>Trespasser</i> = 1	67,42%
3.	<i>Intrusion</i> = 0 dan <i>Trespasser</i> = 1	63,25%
4.	<i>Intrusion</i> = 1 dan <i>Trespasser</i> = 3	73,71%
5.	<i>Intrusion</i> = 2 dan <i>Trespasser</i> = 1	75,22%
6.	<i>Intrusion</i> = 3 dan <i>Trespasser</i> = 2	64,65%
7.	<i>Intrusion</i> = 1 dan <i>Trespasser</i> = 3	71,29%

Tabel 1. Nilai Akurasi

Kesimpulan, pengujian ini mampu mengklarifikasi *intrusion* dan *trespasser* pada jalur kereta api dengan tingkat akurasi tertentu. Penggunaan metode YOLOv8n dan *Canny Edge Detection* efektif dalam mendeteksi serta mengklasifikasikan objek di sepanjang jalur rel. Akan tetapi, nilai akurasi deteksi objek masih belum sempurna. Karena kurangnya jumlah dataset dan variasi dataset yang digunakan dalam proses pelatihan model. Oleh karena itu, diperlukan peningkatan jumlah dan variasi dataset sehingga dapat meningkatkan akurasi, keselamatan, dan mengurangi kecelakaan pada transportasi kereta api.

REFERENCES

- [1] A. Fadjeri, E. Burhaein, and I. P. Widiyono, "Pengembangan Teknologi AI berbasis Computer Vision dan Robotika untuk Latihan Akurasi Olahraga Tennis Lapangan Duduk bagi Atlet Disabilitas Fisik," no. 1, 2025.
- [2] O. Alfiano and S. Rahayu, "IMPLEMENTASI ALGORITMA DEEP LEARNING YOLO (YOU ONLY LOOK ONCE) UNTUK DETEKSI KUALITAS KENTANG SEGAR DAN BUSUK SECARA REAL TIME," *Journal of Research and Publication Innovation*, vol. 2, no. 3, pp. 2470–2478, Jul. 2024.
- [3] S. Ning, F. Ding, and B. Chen, "Research on the Method of Foreign Object Detection for Railway Tracks Based on Deep Learning," *Sensors*, vol. 24, no. 14, p. 4483, Jul. 2024, doi: 10.3390/s24144483.
- [4] W. Baron, "Trespasser Detection Systems on Railroad Rights-of-Way".
- [5] I. Maulana, N. Rahaningsih, and T. Suprpti, "ANALISIS PENGGUNAAN MODEL YOLOV8 (YOU ONLY LOOK ONCE) TERHADAP DETEKSI CITRA SENJATA BERBAHAYA," vol. 7, no. 6, 2023.
- [6] A. A. Zaidan, "Image Enhancement Serbaguna dengan Gabungan Histogram Equalization Global dan Adaptif serta Deteksi Tepi".
- [7] M. H. V. Sinaga, M. Albirra, and M. F. Sidiq, "Klasifikasi Gambar Pemandangan dengan Kecerdasan Buatan Berbasis CNN," *jtik*, vol. 8, no. 2, pp. 412–417, Apr. 2024, doi: 10.35870/jtik.v8i2.1424.

- [8] N. J. Hayati, D. Singasatia, and M. R. Muttaqin, "Object Tracking Menggunakan Algoritma You Only Look Once (YOLO)v8 untuk Menghitung Kendaraan," *KOMPUTA*, vol. 12, no. 2, pp. 91–99, Nov. 2023, doi: 10.34010/komputa.v12i2.10654.
- [9] M. R. Qisthiano and A. O. Pratiwi, "DETEKSI TEPI PADA CITRA OBJEK BENDA MENGGUNAKAN ALGORITMA SOBEL DAN PREWITT DENGAN PYTHON," *JITET*, vol. 13, no. 2, Apr. 2025, doi: 10.23960/jitet.v13i2.6407.